SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:100

Annexure A

Constraint Based Problem Solving

Code of Conduct:

I M. Vidhya (192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

SOL)

Constraint-Based Data Cleaning Pipeline

A large patient health-record dataset stored in a CSV file may contain missing values, inconsistent formats, duplicate records, and outliers. Using Pandas and NumPy, a constraint-based cleaning pipeline can be designed to ensure that the data is complete, medically valid, and standardized.

Step 1: Load and inspect the dataset

```
import pandas as pd

import numpy as np

df = pd.read_csv("patients.csv")

print(df.head())

print(df.info())

print(df.isnull().sum())
```

First, the dataset is loaded using Pandas. The info() function checks column data types, while isnull().sum() identifies the number of missing values in each column.

Step 2: Handle missing values

The first constraint is that no missing values should remain. Numerical columns can be filled using the median, while categorical columns can be filled using the mode.

```
df["Age"] = df["Age"].fillna(df["Age"].median())

df["BloodPressure"] = df["BloodPressure"].fillna(

    df["BloodPressure"].median()

)

df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
```

Median is preferred for numerical medical data because it is less affected by extreme values.

Step 3: Check medically valid numerical ranges

The second constraint is that all numerical values must fall within medically valid ranges. Filtering and indexing can be used to find invalid values.

For example:

```
df.loc[(df["Age"] < 0) | (df["Age"] > 120), "Age"] = np.nan
```

After identifying invalid values, they can be replaced with an appropriate median value.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Similar constraints can be applied to blood pressure, heart rate, glucose level, cholesterol, and other medical measurements according to their valid medical ranges.

Step 4: Handle outliers

Outliers should be checked before removing them because some extreme medical values may represent genuine patient conditions. Therefore, instead of blindly deleting them, they can be identified using statistical methods such as the IQR method.

```
Q1 = df["Glucose"].quantile(0.25)

Q3 = df["Glucose"].quantile(0.75)

IQR = Q3 - Q1

outliers = df[
    (df["Glucose"] < Q1 - 1.5 * IQR) |
    (df["Glucose"] > Q3 + 1.5 * IQR)
]
```

The identified records can then be investigated and corrected or retained depending on whether they are genuine medical observations.

Step 5: Standardize categorical fields

The third constraint is that categorical values should have a consistent format.

```
df["Gender"] = df["Gender"].str.strip().str.lower()

df["Gender"] = df["Gender"].replace({
    "m": "male",
    "f": "female"
})
```

This converts values such as M, m, and Male into the same standardized value, male.

Step 6: Use indexing and filtering

Indexing is used to select specific rows or columns.

```
df.loc[df["Age"] > 60, ["Age", "Gender", "BloodPressure"]]
```

Filtering is used to identify records satisfying particular constraints.

```
valid = df[
   (df["Age"] >= 0) &
   (df["Age"] <= 120)
]
```

These operations allow invalid records to be detected without unnecessarily deleting valid patient information.

Step 7: Use aggregation for validation

Aggregation functions such as count(), mean(), median(), min(), and max() can be used to verify the cleaned dataset.

```
print(df["Age"].min())
print(df["Age"].max())
print(df["Age"].median())
print(df.isnull().sum())
```

Grouping can also be used to check categorical data:

```
print(df.groupby("Gender")["Age"].mean())
```

Step 8: Final validation

After cleaning, the constraints must be checked again.

```
print(df.isnull().sum())
print(df["Age"].between(0, 120).all())
print(df["Gender"].unique())
```

The final dataset should have no missing values, all numerical values should be within the defined medically valid ranges, and categorical fields should follow a common format.

Conclusion

Thus, a Pandas and NumPy constraint-based data cleaning pipeline uses indexing, filtering, replacement, outlier detection, standardization, and aggregation to clean patient records. Instead of deleting large numbers of records, invalid or missing values are carefully corrected

wherever possible, thereby preserving maximum data integrity while satisfying all the required constraints.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

SOL)

Memory-Efficient Processing of a Massive Numerical Dataset Using NumPy

A massive numerical dataset may require a large amount of memory during mathematical operations, random sampling, and statistical analysis. To avoid memory overflow, NumPy arrays, efficient data types, slicing, indexing, aggregation, and in-place operations can be used.

Step 1: Create memory-efficient arrays

NumPy arrays should be created with a suitable data type. If very high precision is not required, float32 can be used instead of float64.

```
import numpy as np

data = np.zeros(1000000, dtype=np.float32)
```

Using a smaller data type reduces the amount of memory occupied by the array.

Step 2: Use slicing instead of copying

Slicing can select only the required portion of a large array.

```
sample = data[::10]
```

This avoids creating an unnecessary full copy of the dataset and therefore reduces memory usage.

Step 3: Perform mathematical operations in-place

In-place operations modify the existing array instead of creating a new array.

```
data *= 2

data += 5
```

This is more memory-efficient than creating separate arrays for every operation.

Step 4: Perform random sampling efficiently

Instead of copying the complete dataset, only the required indices can be selected.

```
indices = np.random.choice(
    len(data), size=1000, replace=False
)
sample = data[indices]
```

This allows a small random sample to be processed without unnecessarily loading additional data.

Step 5: Perform statistical analysis

NumPy aggregation functions can calculate statistical values efficiently.

```
mean = np.mean(data)
maximum = np.max(data)
minimum = np.min(data)
variance = np.var(data)
```

These vectorized functions are faster than manually processing elements using Python loops.

Step 6: Optimize indexing

Boolean indexing can be used to select only the required values.

```
positive_data = data[data > 0]
```

However, for very large arrays, unnecessary boolean copies should be avoided. If only the result of an aggregation is required, the calculation can be performed directly.

```
mean_positive = np.mean(data[data > 0])
```

The dataset should also be processed in chunks when it is too large to fit into memory at once.

Step 7: Optimize aggregation

Aggregation should be performed directly on the required portion of the array rather than creating several intermediate arrays.

For example:

```
mean = np.mean(data)
std = np.std(data)
```

Unnecessary intermediate calculations and repeated conversions should be avoided.

Debugging memory overflow and performance problems

If memory overflow occurs:

- Check the memory occupied by the array using data.nbytes.
- Use smaller data types such as float32 where appropriate.
- Avoid unnecessary .copy() operations.
- Use slicing and chunk-based processing.
- Delete unused large arrays using del.
- Avoid creating multiple temporary arrays.

If the program is slow:

- Replace Python loops with NumPy vectorized operations.
- Check which operation is consuming most of the processing time.
- Reduce unnecessary indexing and repeated aggregation.
- Process only the required portion of the dataset.

Conclusion

Thus, efficient array creation, appropriate data types, slicing, indexing, random sampling, vectorized mathematical operations, aggregation, and in-place operations allow massive numerical datasets to be processed efficiently without exceeding memory limits. Debugging memory usage and identifying performance bottlenecks further improves the reliability and speed of the system.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

SOL)

Combining Two Large Pandas DataFrames

Two large Pandas DataFrames containing customer transaction data and customer demographic data can be combined using Pandas merge() while satisfying the given

constraints: no duplication of customer IDs, consistent indexing, and preservation of all valid records.

Step 1: Check the DataFrames

First, load both DataFrames and inspect their structure.

```
import pandas as pd

transactions = pd.read_csv("transactions.csv")

demographic = pd.read_csv("demographic.csv")

print(transactions.info())

print(demographic.info())
```

This helps identify column names, data types, missing values, and possible inconsistencies.

Step 2: Check for duplicate Customer IDs

Since there should be no duplication of customer IDs, check both DataFrames before combining them.

```
print(transactions["CustomerID"].duplicated().sum())

print(demographic["CustomerID"].duplicated().sum())
```

If duplicate IDs are found, they should be investigated. Duplicate records should only be removed when they represent actual duplicate records, so that valid transaction information is not lost.

Step 3: Combine the DataFrames

The DataFrames can be combined using merge() based on the common CustomerID.

```
df = pd.merge(
    transactions,
    demographic,
    on="CustomerID",
    how="outer"
)
```

An outer join is suitable when the requirement is to preserve all valid records from both DataFrames. It keeps customers even if their information is missing from one of the DataFrames.

Step 4: Validate the Customer IDs

After merging, use filtering and grouping to check whether customer IDs have been duplicated.

```
duplicates = df[
    df["CustomerID"].duplicated(keep=False)
]


print(duplicates)
```

Grouping can also be used:

```
print(df.groupby("CustomerID").size())
```

If a customer ID appears more than once unexpectedly, the corresponding records should be investigated.

Step 5: Check missing and mismatched records

Missing values can be identified using:

```
print(df.isnull().sum())
```

To identify whether a customer exists only in one DataFrame, use the merge indicator.

```
check = pd.merge(
    transactions,
    demographic,
    on="CustomerID",
    how="outer",
    indicator=True
)
print(check["_merge"].value_counts())
```

The _merge column identifies records that are present in both DataFrames or only in one of them.

Step 6: Ensure consistent indexing

After merging, sort the records by CustomerID and reset the index.

```
df = df.sort_values("CustomerID")
df = df.reset_index(drop=True)
```

This provides a clean and consistent index.

Step 7: Verify correctness

The final DataFrame should be checked for duplicate IDs and missing customer IDs.

print(df["CustomerID"].duplicated().sum())

print(df["CustomerID"].isnull().sum())

Sorting also makes it easier to identify missing or incorrectly ordered customer records.

Debugging mismatched entries

If records do not match:

- Check whether CustomerID has the same data type in both DataFrames.
- Remove unwanted spaces from IDs.
- Check for spelling or formatting differences.
- Identify IDs present in only one DataFrame using indicator=True.
- Check missing values before merging.

For example:

transactions["CustomerID"] = transactions["CustomerID"].astype(str).str.strip()

demographic["CustomerID"] = demographic["CustomerID"].astype(str).str.strip()

Conclusion

Thus, merge(), filtering, grouping, sorting, and indexing can be used to combine large Pandas DataFrames correctly. An outer merge preserves all valid records, while duplicate checking and grouping ensure that customer IDs are not unnecessarily duplicated. Finally, sorting and resetting the index provide a consistent and organized DataFrame.

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

SOL)

Computing Statistical Measures Using NumPy and Pandas

The required statistical measures are mean, median, and variance. They can be calculated efficiently and accurately using NumPy and Pandas while avoiding unnecessary computations and precision loss.

Step 1: Load and inspect the dataset

```
import pandas as pd

import numpy as np

df = pd.read_csv("data.csv")

print(df.info())

print(df.isnull().sum())
```

First, check the data type, missing values, and irrelevant records before performing statistical calculations.

Step 2: Use filtering to remove irrelevant data

Filtering can be used to select only valid data points.

```
df = df[df["Value"].notna()]

df = df[df["Value"] >= 0]
```

This prevents missing or invalid values from affecting the statistical results.

Step 3: Use indexing to select required data

Only the required column should be selected for calculation.

```
x = df.loc[:, "Value"]
```

This avoids processing unnecessary columns and improves computational efficiency.

Step 4: Calculate mean, median and variance

Using Pandas:

```
mean = x.mean()

median = x.median()

variance = x.var()
```

Using NumPy:

```
mean = np.mean(x)

median = np.median(x)

variance = np.var(x)
```

These built-in aggregation functions are faster and more efficient than calculating each value using Python loops.

Step 5: Avoid precision loss

For numerical accuracy, an appropriate data type should be used.

x = x.to_numpy(dtype=np.float64)

float64 provides better precision for statistical calculations when compared with lower-precision data types.

It is also important to avoid repeatedly converting the same data or performing the same calculation multiple times.

Step 6: Avoid redundant computations

The data should be filtered and converted only once before calculating the required measures.

x = df.loc[df["Value"].notna(), "Value"].to_numpy(dtype=np.float64)

mean = np.mean(x)

median = np.median(x)

variance = np.var(x)

This reduces unnecessary processing.

Step 7: Check statistical accuracy

The results should be verified by checking the basic properties of the data.

print("Minimum:", np.min(x))

print("Maximum:", np.max(x))

print("Mean:", np.mean(x))

print("Median:", np.median(x))

print("Variance:", np.var(x))

The minimum and maximum values help identify unexpected values that may affect the result.

Debugging incorrect statistical outputs

If the statistical results are incorrect:

- Check for missing values using isnull().sum().
- Check whether irrelevant or invalid values are included.
- Verify the data type of the column.
- Check the filtering conditions.
- Compare NumPy and Pandas results.
- Check whether sample variance or population variance is required.
- Check for extreme outliers that may affect the mean and variance.

For example, Pandas var() uses sample variance by default, while NumPy np.var() uses population variance by default. Therefore, the correct option must be selected according to the question.

Conclusion

Thus, filtering and indexing are used to select valid and relevant data, while NumPy and Pandas aggregation functions efficiently calculate mean, median, and variance. Using suitable numerical precision and avoiding repeated calculations ensures both computational efficiency and numerical accuracy.

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

SOL)

Complete Data Analysis Pipeline

A complete data analysis pipeline consists of loading, cleaning, filtering, grouping, sorting, aggregation, and visualization.

Step 1: Load multiple files efficiently

```
import pandas as pd
import glob
files = glob.glob("data/*.csv")
df = pd.concat(
    [pd.read_csv(file) for file in files],
    ignore_index=True
)
```

Multiple CSV files are loaded and combined into one DataFrame.

Step 2: Check and clean the data

```
print(df.info())
```

```
print(df.isnull().sum())
```

Correct data types, missing values, and inconsistent formats before analysis.

Step 3: Apply user-defined filtering

```
filtered = df[df["Sales"] > 1000]
```

Only records satisfying the user's condition are selected.

Step 4: Group and aggregate

```
summary = (
    filtered.groupby("Category")["Sales"]
    .sum()
    .reset_index()
)
```

Grouping calculates useful summaries for each category.

Step 5: Sort the results

```
summary = summary.sort_values(
    "Sales", ascending=False
)
```

Sorting makes the results easier to interpret.

Step 6: Generate meaningful visualization

```
import matplotlib.pyplot as plt
plt.bar(summary["Category"], summary["Sales"])
plt.xlabel("Category")
plt.ylabel("Sales")
plt.title("Sales by Category")
plt.xticks(rotation=45)
plt.show()
```

The graph should clearly represent the required relationship.

Optimization and debugging:

- Read only required columns using usecols.
- Specify suitable data types using dtype.

- Process very large files in chunks using chunksize.
- Check column names and data types before processing.
- Handle corrupted files using try-except.
- Check for missing and duplicate records.
- Verify grouping and sorting results before plotting.
- Check whether the graph contains meaningful and correctly labeled data.

Conclusion:

The pipeline efficiently converts multiple raw files into clean, filtered, aggregated, sorted, and meaningful visual information while providing debugging steps for corrupted or inconsistent data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand